

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ ИНФОРМАТИКИ

Отчет по лабораторной работе №5,6,7
по курсу «Алгоритмы и структуры данных»
Тема: Деревья. Пирамида, пирамидальная сортировка.
Очередь с приоритетами. Хеширование. Хеш-таблицы.
Динамическое программирование №1
Вариант 4

Выполнила:
Баранова Софья
К3162

Проверил:
Аминов Натиг Сабит оглы

Санкт-Петербург
2025 г.

Содержание отчета

Содержание отчета	2
Лабораторная работа №5	3
Задачи по варианту	3
Задача №1. Куча ли?	3
Задача №5. Планировщик заданий	6
Дополнительные задачи	11
Задача №2. Высота дерева	11
Вывод	14
Лабораторная работа №6	15
Задачи по варианту	15
Задача №1. Множество	15
Задача №2. Телефонная книга	18
Дополнительные задачи	21
Задача №5. Выборы в США	21
Вывод	24
Лабораторная работа №7	25
Задачи по варианту	25
Задача №4. Наибольшая общая подпоследовательность двух последовательностей	25
Задача №6. Наибольшая возрастающая подпоследовательность	27
Дополнительные задачи	31
Задача №1. Обмен монет	31
Вывод	34

Лабораторная работа №5

Задачи по варианту

Задача №1. Куча ли?

Структуру данных «куча», или, более конкретно, «неубывающая пирамида», можно реализовать на основе массива. Для этого должно выполняться основное свойство неубывающей пирамиды, которое заключается в том, что для каждого $1 \leq i \leq n$ выполняются условия:

1. если $2i \leq n$, то $a_i \leq a_{2i}$,
2. если $2i + 1 \leq n$, то $a_i \leq a_{2i+1}$.

Дан массив целых чисел. Определите, является ли он неубывающей пирамидой.

- Формат входного файла (input.txt). Первая строка входного файла содержит целое число n ($1 \leq n \leq 10^6$). Вторая строка содержит n целых чисел, по модулю не превосходящих $2 \cdot 10^9$
- Формат выходного файла (output.txt). Выведите «YES», если массив является неубывающей пирамидой, и «NO» в противном случае.

```
import time
import tracemalloc
tracemalloc.start()
st = time.perf_counter()

def is_heap(a, n):
    for i in range(1, n // 2 + 1):
        left = 2 * i
        right = 2 * i + 1

        if left <= n and a[i] > a[left]:
            return False
        if right <= n and a[i] > a[right]:
            return False

    return True

def main():
    with open("input.txt", "r") as f:
        n = int(f.readline())
        numbers = list(map(int, f.readline().split()))

    a = [0] + numbers

    result = "YES" if is_heap(a, n) else "NO"

    with open("output.txt", "w") as f:
        f.write(result)
```

```

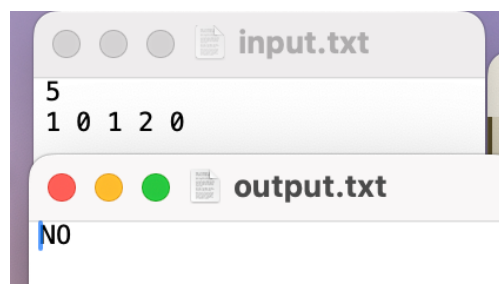
main()

print('Время работы: %s секунд' % (time.perf_counter() - st))
pick = tracemalloc.get_traced_memory()[1]
tracemalloc.stop()
print("Объем используемой памяти: " + str(pick/1024) + 'Кбайт')

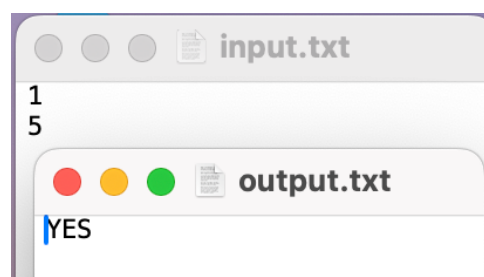
```

Импортируем модули для замера используемой памяти и времени. Из файла `input.txt` считывается число `n` и последовательность из `n` чисел, которая рассматривается как массив, представляющий бинарную кучу. Для удобства индексации к массиву добавляется фиктивный нулевой элемент, чтобы элементы кучи нумеровались с единицы. Последовательность передаётся в функцию `is_heap`, которая проверяет, является ли она корректной минимальной кучей. Для каждой внутренней вершины `i` функция смотрит на её левого и правого потомков (индексы $2 \cdot i$ и $2 \cdot i + 1$) и проверяет, что значение в вершине не больше значений в потомках. Если где-то это условие нарушается, функция возвращает `False`. Если нарушений нет, возвращается `True`. В зависимости от результата в файл `output.txt` записывается «YES», если последовательность задаёт корректную минимальную кучу, и «NO» — в противном случае. В конце программа выводит время выполнения и объём использованной оперативной памяти.

Результат работы кода на примерах из текста задачи:(скрины input output файлов)



Результат работы кода на минимальных значениях:(скрины input output файлов)



Результат работы кода на максимальных значениях:(скрины input output файлов)

```

input.txt
1000000
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34
35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50
51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66
67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82
83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98
99 100 101 102 103 104 105 106 107 108 109 110
111 112 113 114 115 116 117 118 119 120 121 122
123 124 125 126 127 128 129 130 131 132 133 134
135
147
159 YES
171
183
195
207
219
231
243
255
267
279

output.txt
YES
  
```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.000210с	13.8173 Кбайт
Пример из задачи	0.000254с	13.9414 Кбайт
Верхняя граница диапазона значений входных данных из текста задачи	0.761293с	89664.0117 Кбайт

Вывод по задаче:

Программа эффективно проверяет массив на соответствие структуре минимальной кучи, опираясь на прямой доступ к элементам и простые соотношения индексов родитель–потомок. Последовательная проверка только нужной части массива делает алгоритм быстрым и надёжным, без лишних вычислений и обходов.

Задача №5. Планировщик заданий

В этой задаче вы создадите программу, которая параллельно обрабатывает список заданий. Во всех операционных системах, таких как Linux, MacOS или Windows, есть специальные программы, называемые планировщиками, которые делают именно это с программами на вашем компьютере. У вас есть программа, которая распараллеливается и использует n независимых потоков для обработки заданного списка m заданий. Потоки берут задания в том порядке, в котором они указаны во входных данных. Если есть свободный поток, он немедленно берет следующее задание из списка. Если поток начал обработку задания, он не прерывается и не останавливается, пока не завершит обработку задания. Если несколько потоков одновременно пытаются взять задания из списка, поток с меньшим индексом берет задание. Для каждого задания вы точно знаете, сколько времени потребуется любому потоку, чтобы обработать это задание, и это время одинаково для всех потоков. Вам необходимо определить для каждого задания, какой поток будет его обрабатывать и когда он начнет обработку.

- **Формат ввода или входного файла (input.txt).** Первая строка содержит целые числа n и m ($1 \leq n \leq 105$, $1 \leq m \leq 105$). Вторая строка содержит m целых чисел t_i - время в секундах, которое требуется для выполнения i -ой задания любым потоком ($0 \leq t_i \leq 109$). Все эти значения даны в том порядке, в котором они подаются на выполнение. Индексы потоков начинаются с 0.
- **Формат выходного файла (output.txt).** Выведите в точности m строк, причем i -ая строка (начиная с 0) должна содержать два целочисленных значения: индекс потока, который выполняет i -ое задание, и время в секундах, когда этот поток начал выполнять задание.

```
import time
import tracemalloc
tracemalloc.start()
st = time.perf_counter()
```

```
def sift_down(heap, i):
    n = len(heap)
    while True:
        left = 2 * i + 1
        right = 2 * i + 2
        smallest = i
```

```
        if left < n and heap[left] < heap[smallest]:
            smallest = left
        if right < n and heap[right] < heap[smallest]:
            smallest = right
```

```

        if smallest == i:
            break

    heap[i], heap[smallest] = heap[smallest], heap[i]
    i = smallest

def sift_up(heap, i):
    while i > 0:
        parent = (i - 1) // 2
        if heap[i] < heap[parent]:
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break

def heap_push(heap, value):
    heap.append(value)
    sift_up(heap, len(heap) - 1)

def heap_pop(heap):
    heap[0], heap[-1] = heap[-1], heap[0]
    value = heap.pop()
    if heap:
        sift_down(heap, 0)
    return value

def solve(n, m, times):
    heap = [(0, i) for i in range(n)]

    for i in reversed(range(n // 2)):
        sift_down(heap, i)

    results = []

    for t in times:
        free_time, thread_id = heap_pop(heap)
        results.append(f"{thread_id} {free_time}")

        heap_push(heap, (free_time + t, thread_id))

    return results

def main():
    with open("input.txt", "r") as f:
        data = list(map(int, f.read().split()))

    n, m = data[0], data[1]
    times = data[2:2 + m]

    results = solve(n, m, times)

```

```

with open("output.txt", "w") as f:
    f.write("\n".join(results))

if __name__ == "__main__":
    main()

print('Время работы: %s секунд' % (time.perf_counter() - st))
pick = tracemalloc.get_traced_memory()[1]
tracemalloc.stop()
print("Объем используемой памяти: " + str(pick/1024) + 'Кбайт')

```

Импортируем модули для замера используемой памяти и времени. Из файла input.txt считываются количество потоков n , количество задач m и времена выполнения каждой задачи.

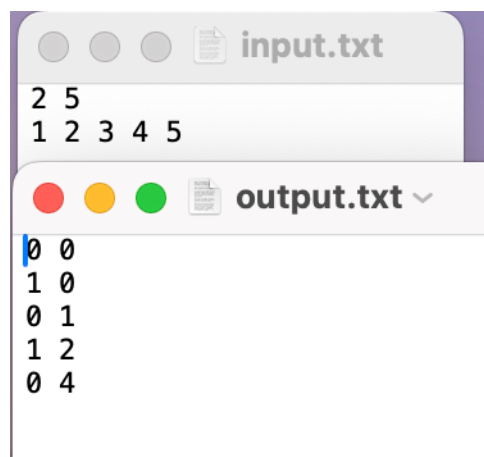
Данные передаются в функцию solve, где моделируется работа пула потоков с помощью **минимальной кучи**. В куче хранятся пары (время, когда поток освободится, номер потока). Сначала все потоки считаются свободными в момент времени 0.

Для каждой задачи из списка:

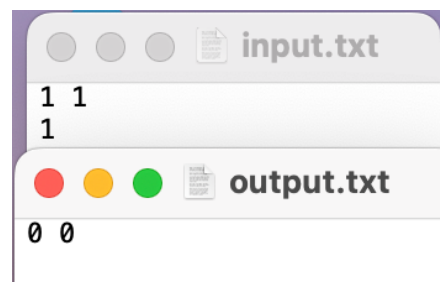
- из кучи извлекается поток, который освободится раньше всех (при равенстве времени — с меньшим номером);
- в результат записывается номер потока и время начала выполнения задачи (текущее время освобождения потока);
- поток возвращается в кучу уже с обновлённым временем освобождения: к старому времени прибавляется длительность текущей задачи.

После обработки всех задач программа записывает в файл output.txt для каждой задачи строку вида «номер_потока время_начала». В конце выводится время выполнения и объём использованной оперативной памяти.

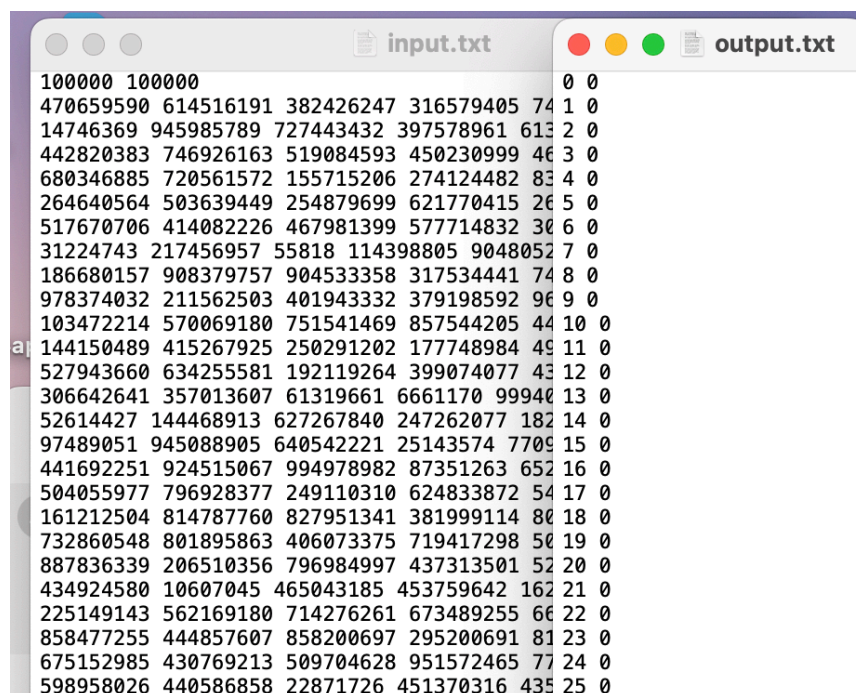
Результат работы кода на примерах из текста задачи: (скрины input output файлов)



Результат работы кода на минимальных значениях:(скрины input output файлов)



Результат работы кода на максимальных значениях:(скрины input output файлов)



	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.000316с	7.4726 Кбайт
Пример из задачи	0.000230с	7.7070Кбайт

Верхняя граница диапазона значений входных данных из текста задачи	1.244346с	22250.7480 Кбайт
---	-----------	------------------

Вывод по задаче:

Программа эффективно моделирует распределение задач между потоками, используя минимальную кучу для выбора потока, который освободится раньше всех. Такая структура данных обеспечивает быстрые операции вставки и извлечения, что позволяет обрабатывать до ста тысяч задач при строгих временных ограничениях. Реализация остаётся компактной и надёжной, гарантируя корректное распределение работы даже на максимальных входных данных.

Дополнительные задачи

Задача №2. Высота дерева

В этой задаче ваша цель - привыкнуть к деревьям. Вам нужно будет прочитать описание дерева из входных данных, реализовать структуру данных, сохранить дерево и вычислить его высоту.

- Вам дается корневое дерево. Ваша задача - вычислить и вывести его высоту. Напомним, что высота (корневого) дерева - это максимальная глубина узла или максимальное расстояние от листа до корня. Вам дано произвольное дерево, не обязательно бинарное дерево.
- **Формат ввода или входного файла (input.txt).** Первая строка содержит число узлов n ($1 \leq n \leq 105$). Вторая строка содержит n целых чисел от -1 до $n-1$ – указание на родительский узел. Если i -ое значение равно -1 , значит, что узел i - корневой, иначе это число является обозначением индекса родительского узла этого i -го узла ($0 \leq i \leq n - 1$). Индексы считать с 0. Гарантируется, что дан только один корневой узел, и что входные данные представляют дерево.
- **Формат вывода или выходного файла (output.txt).** Выведите целое число – высоту данного дерева.

```
import time
import tracemalloc
tracemalloc.start()
st = time.perf_counter()

from collections import deque

def tree_height(parents):
    n = len(parents)
    children = [[] for _ in range(n)]
    root = -1

    for i, p in enumerate(parents):
        if p == -1:
            root = i
        else:
            children[p].append(i)

    height = 0
    q = deque()
    q.append((root, 1))

    while q:
        v, d = q.popleft()
        height = max(height, d)
        for u in children[v]:
            q.append((u, d + 1))
```

```

    return height

def main():
    with open('input.txt', 'r', encoding='utf-8') as f:
        data = f.read().split()

    n = int(data[0])
    parents = list(map(int, data[1:1 + n]))

    h = tree_height(parents)

    with open('output.txt', 'w', encoding='utf-8') as f:
        f.write(str(h))

if __name__ == "__main__":
    main()

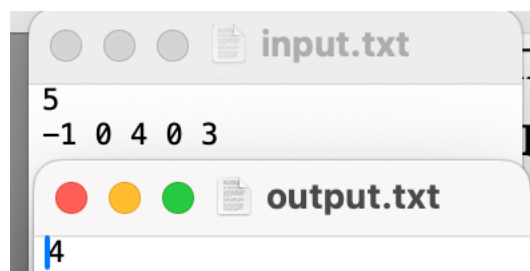
print('Время работы: %s секунд' % (time.perf_counter() - st))
pick = tracemalloc.get_traced_memory()[1]
tracemalloc.stop()
print("Объем используемой памяти: " + str(pick/1024) + 'Кбайт')

```

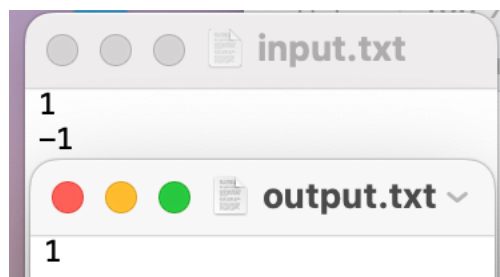
Из файла input.txt считывается число n и массив из n элементов, где каждый элемент — это номер родителя соответствующей вершины дерева (или -1 , если вершина является корнем). Эти данные передаются в функцию `tree_height`, которая вычисляет высоту дерева.

Сначала программа строит список детей для каждой вершины и находит корневую вершину. Затем выполняется обход дерева в ширину (BFS), начиная с корня: в очередь помещаются пары (вершина, текущая глубина). На каждом шаге глубина обновляется, и для всех детей вершины глубина увеличивается на 1. Максимальная достигнутая глубина и есть высота дерева. Полученное значение высоты записывается в файл output.txt. В конце программа выводит время выполнения и объём использованной оперативной памяти.

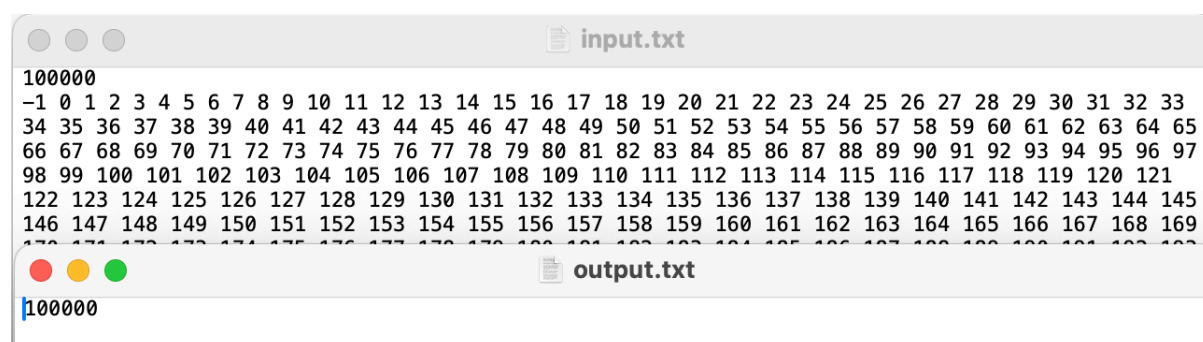
Результат работы кода на примерах из текста задачи: (скрины input output файлов)



Результат работы кода на минимальных значениях:(скрины input output файлов)



Результат работы кода на максимальных значениях:(скрины input output файлов)



	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.000194с	6.8076 Кбайт
Пример из задачи	0.000260с	6.8623 Кбайт
Верхняя граница диапазона значений входных данных из текста задачи	0.17486с	20878.8964 Кбайт

Вывод по задаче:

Программа корректно и эффективно вычисляет высоту дерева, используя обход в ширину. Решение работает за линейное время и подходит для максимальных ограничений задачи. Реализация наглядна и демонстрирует умение работать со структурами данных и графами.

Вывод

В ходе лабораторной работы были реализованы и протестированы структуры данных, основанные на бинарной куче и деревьях. Задачи показали, что правильная организация данных напрямую влияет на эффективность алгоритмов: при проверке свойств кучи достаточно линейного прохода, а использование минимальной кучи в моделировании планировщика обеспечивает оптимальное распределение ресурсов при больших объёмах входных данных. Также был реализован обход дерева в ширину для вычисления его высоты, что продемонстрировало важность выбора корректного метода обхода структуры. Все решения масштабируются до крайних значений входных данных, сохраняя корректность и высокую скорость работы.

Лабораторная работа №6

Задачи по варианту

Задача №1. Множество

Реализуйте множество с операциями «добавление ключа», «удаление ключа», «проверка существования ключа».

- **Формат входного файла (input.txt).** В первой строке входного файла находится строго положительное целое число операций N , не превышающее $5 \cdot 10^5$. В каждой из последующих N строк находится одна из следующих операций:

- $A\ x$ – добавить элемент x в множество. Если элемент уже есть в множестве, то ничего делать не надо.
- $D\ x$ – удалить элемент x . Если элемента x нет, то ничего делать не надо.
- $?\ x$ – если ключ x есть в множестве, выведите «Y», если нет, то выведите «N».

Аргументы указанных выше операций – целые числа, не превышающие по модулю 10^{18} .

- **Формат выходного файла (output.txt).** Выведите последовательно результат выполнения всех операций «?». Следуйте формату выходного файла из примера.

- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.

```
import time
import tracemalloc
tracemalloc.start()
st = time.perf_counter()

def main():
    with open('input.txt', 'r') as f:
        first_line = f.readline()
        n = int(first_line)
        s = set()
        ans = []

        for _ in range(n):
            line = f.readline().split()
            op, x = line[0], int(line[1])

            if op == 'A':
                s.add(x)
            elif op == 'D':
                s.discard(x)
            elif op == '?':
```

```

        ans.append('Y' if x in s else 'N')

    with open('output.txt', 'w') as f:
        f.write('\n'.join(ans))

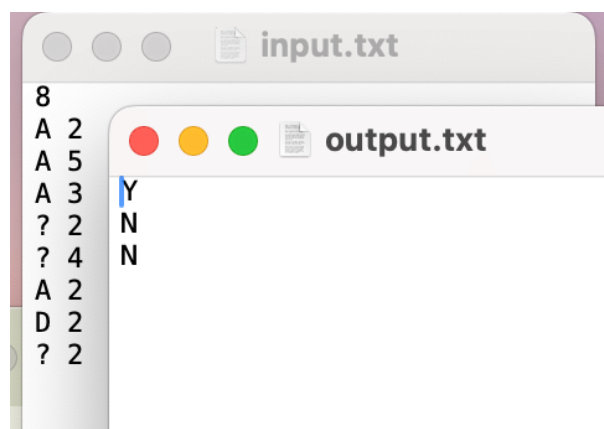
if __name__ == '__main__':
    main()

print('Время работы: %s секунд' % (time.perf_counter() - st))
pick = tracemalloc.get_traced_memory()[1]
tracemalloc.stop()
print("Объем используемой памяти: " + str(pick/1024) + 'Кбайт')

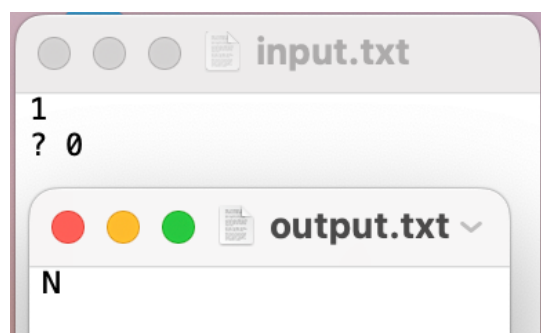
```

Из файла input.txt считывается число n - количество операций, затем n строк с командами работы с множеством целых чисел. Во время обработки команд используется множество s. После выполнения всех операций результаты всех запросов типа ? построчно записываются в файл output.txt. В конце программа выводит время выполнения и объём использованной оперативной памяти.

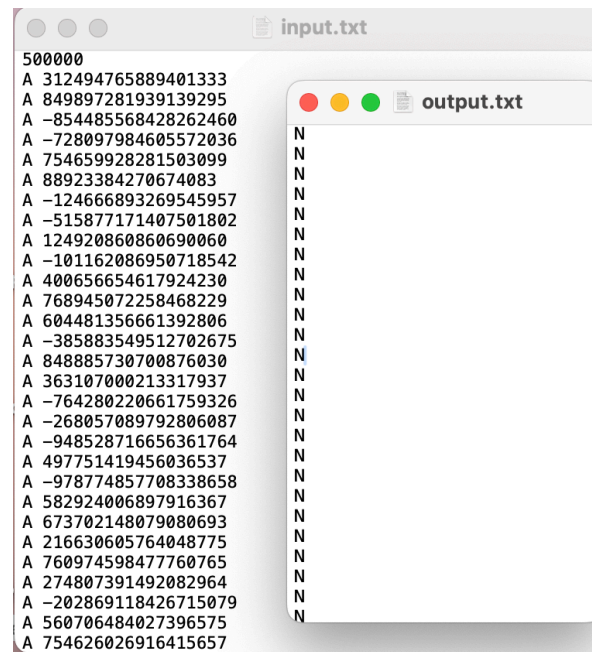
Результат работы кода на примерах из текста задачи:(скрины input output файлов)



Результат работы кода на минимальных значениях:(скрины input output файлов)



Результат работы кода на максимальных значениях:(скрины input output файлов)



	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.000208с	13.6611Кбайт
Пример из задачи	0.000297с	14.1503 Кбайт
Верхняя граница диапазона значений входных данных из текста задачи	0.610327с	18198.5957 Кбайт

Вывод по задаче:

Программа эффективно обрабатывает большое количество операций над множеством, позволяя быстро добавлять, удалять элементы и проверять их наличие. Реализация проста, опирается на встроенную структуру данных `set`, которая обеспечивает высокую производительность даже при максимальных ограничениях задачи.

Задача №2. Телефонная книга

В этой задаче ваша цель - реализовать простой менеджер телефонной книги. Он должен уметь обрабатывать следующие типы пользовательских запросов:

- `add number name` – это команда означает, что пользователь добавляет в телефонную книгу человека с именем `name` и номером телефона `number`. Если пользователь с таким номером уже существует, то ваш менеджер должен перезаписать соответствующее имя.
- `del number` – означает, что менеджер должен удалить человека с номером из телефонной книги. Если такого человека нет, то он должен просто игнорировать запрос.
- `find number` – означает, что пользователь ищет человека с номером телефона `number`. Менеджер должен ответить соответствующим именем или строкой «not found» (без кавычек), если такого человека в книге нет.
- **Формат ввода / входного файла (input.txt).** В первой строке входного файла содержится число N ($1 \leq N \leq 105$) - количество запросов. Далее следуют N строк, каждая из которых содержит один запрос в формате, описанном выше. Все номера телефонов состоят из десятичных цифр, в них нет нулей в начале номера, и каждый состоит не более чем из 7 цифр. Все имена представляют собой непустые строки из латинских букв, каждая из которых имеет длину не более 15. Гарантируется при проверке, что не будет человека с именем «not found».
- **Формат вывода / выходного файла (output.txt).** Выведите результат каждого поискового запроса `find` – имя, соответствующее номеру телефона, или «not found» (без кавычек), если в телефонной книге нет человека с таким номером телефона. Выведите по одному результату в каждой строке в том же порядке, как были заданы запросы типа `find` во входных данных.

```
import time
import tracemalloc
tracemalloc.start()
st = time.perf_counter()
```

```
def solve(tokens):
    n = int(tokens[0])
    book = {}
```

```
    ans = []
    i = 1
```

```
    for _ in range(n):
```

```

        cmd = tokens[i]
        i += 1

    if cmd == "add":
        number = int(tokens[i])
        name = tokens[i + 1]
        i += 2
        book[number] = name

    elif cmd == "del":
        number = int(tokens[i])
        i += 1
        if number in book:
            del book[number]

    else:
        number = int(tokens[i])
        i += 1
        ans.append(book.get(number, "not found"))

    return "\n".join(ans)

def main():
    with open("input.txt", "r", encoding="utf-8") as f:
        tokens = f.read().split()

    result = solve(tokens)

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write(result)

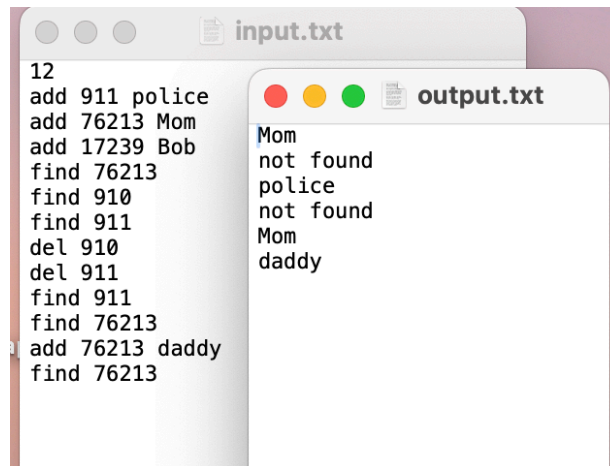
if __name__ == "__main__":
    main()

print('Время работы: %s секунд' % (time.perf_counter() - st))
pick = tracemalloc.get_traced_memory()[1]
tracemalloc.stop()
print("Объем используемой памяти: " + str(pick/1024) + 'Кбайт')

```

Импортируем модули для замера используемой памяти и времени. Из файла input.txt считывается количество операций n, а затем последовательность команд для работы с телефонным справочником. Данные передаются в функцию solve, где с помощью словаря book реализуется структура «номер телефона → имя». После выполнения всех операций результаты всех команд find записываются построчно в файл output.txt. В конце программа выводит время выполнения и объём использованной оперативной памяти.

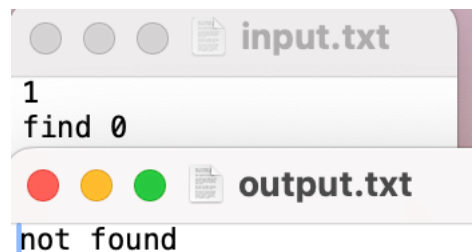
Результат работы кода на примерах из текста задачи:(скрины input output файлов)



```
input.txt
12
add 911 police
add 76213 Mom
add 17239 Bob
find 76213
find 910
find 911
del 910
del 911
find 911
find 76213
add 76213 daddy
find 76213

output.txt
Mom
not found
police
not found
Mom
daddy
```

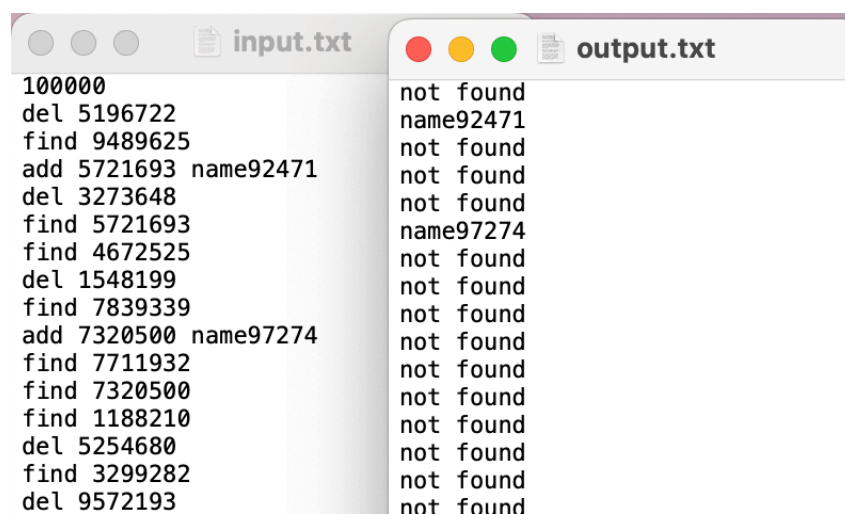
Результат работы кода на минимальных значениях:(скрины input output файлов)



```
input.txt
1
find 0

output.txt
not found
```

Результат работы кода на максимальных значениях:(скрины input output файлов)



```
input.txt
100000
del 5196722
find 9489625
add 5721693 name92471
del 3273648
find 5721693
find 4672525
del 1548199
find 7839339
add 7320500 name97274
find 7711932
find 7320500
find 1188210
del 5254680
find 3299282
del 9572193

output.txt
not found
name92471
not found
not found
not found
name97274
not found
not found
not found
not found
not found
not found
not found
not found
not found
not found
not found
```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.000294с	6.7392 Кбайт
Пример из задачи	0.000237с	8.2763 Кбайт
Верхняя граница диапазона значений входных данных из текста задачи	0.149664с	15424.4082 Кбайт

Вывод по задаче:

Программа надёжно и эффективно выполняет операции с телефонным справочником, используя словарь как хеш-таблицу. Решение обеспечивает быстрый доступ к данным, позволяя оперативно добавлять, удалять и искать записи даже при максимальных размерах входа.

Дополнительные задачи

Задача №5. Выборы в США

Как известно, в США президент выбирается не прямым голосованием, а путем двухуровневого голосования. Сначала проводятся выборы в каждом штате и определяется победитель выборов в данном штате. Затем проводятся государственные выборы: на этих выборах каждый штат имеет определенное число голосов — число выборщиков от этого штата. На практике, все выборщики от штата голосуют в соответствии с результатами голосования внутри штата, то есть на заключительной стадии выборов в голосовании участвуют штаты, имеющие различное число голосов. Вам известно за кого проголосовал каждый штат и сколько голосов было отдано данным штатом. Подведите итоги выборов: для каждого из участника голосования определите число отданных за него голосов.

- **Формат ввода / входного файла (input.txt).** Каждая строка входного файла содержит фамилию кандидата, за которого отдают голоса выборщики этого штата, затем через пробел идет количество выборщиков, отдавших голоса за этого кандидата.

- **Формат вывода / выходного файла (output.txt).** Выведите фамилии всех кандидатов в лексикографическом порядке, затем, через пробел, количество отданных за них голосов.

```
import time
import tracemalloc
tracemalloc.start()
st = time.perf_counter()

def count_votes(lines):
    votes = {}
    for line in lines:
        line = line.strip()
        if not line:
            continue
        name, num = line.split()
        num = int(num)
        if name in votes:
            votes[name] += num
        else:
            votes[name] = num
    return votes

def main():
    with open('input.txt', 'r', encoding='utf-8') as f:
        lines = f.readlines()

    votes = count_votes(lines)

    with open('output.txt', 'w', encoding='utf-8') as f:
        for name in sorted(votes.keys()):
            f.write(f"{name} {votes[name]}\n")

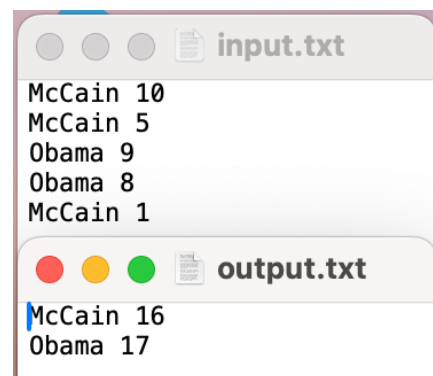
if __name__ == "__main__":
    main()

print('Время работы: %s секунд' % (time.perf_counter() - st))
pick = tracemalloc.get_traced_memory()[1]
tracemalloc.stop()
print("Объем используемой памяти: " + str(pick/1024) + 'Кбайт')
```

Импортируем модули для замера используемой памяти и времени. Из файла input.txt считываются строки, каждая из которых содержит фамилию кандидата и количество голосов, добавившихся к его результату. Эти данные передаются в функцию count_votes, которая с помощью словаря накапливает общее число голосов для каждого кандидата: если кандидат уже встречался, его счёт увеличивается, если нет — создаётся новая запись. После подсчёта голосов программа сортирует фамилии кандидатов по алфавиту и записывает в файл output.txt список вида «фамилия общее

число голосов», по одному кандидату в каждой строке. В конце выводится время выполнения программы и объём использованной оперативной памяти.

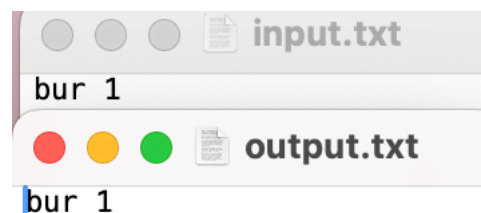
Результат работы кода на примерах из текста задачи:(скрины input output файлов)



```
input.txt
McCain 10
McCain 5
Obama 9
Obama 8
McCain 1

output.txt
McCain 16
Obama 17
```

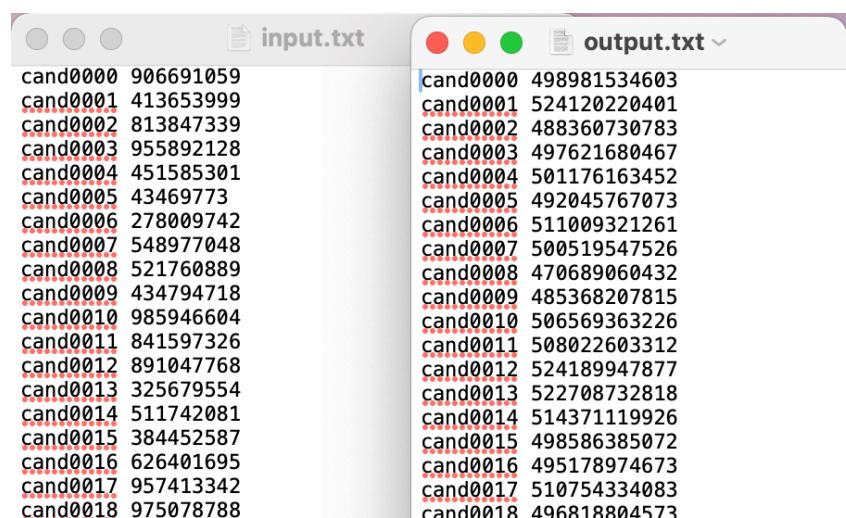
Результат работы кода на минимальных значениях:(скрины input output файлов)



```
input.txt
bur 1

output.txt
bur 1
```

Результат работы кода на максимальных значениях:(скрины input output файлов)



```
input.txt
cand0000 906691059
cand0001 413653999
cand0002 813847339
cand0003 955892128
cand0004 451585301
cand0005 43469773
cand0006 278009742
cand0007 548977048
cand0008 521760889
cand0009 434794718
cand0010 985946604
cand0011 841597326
cand0012 891047768
cand0013 325679554
cand0014 511742081
cand0015 384452587
cand0016 626401695
cand0017 957413342
cand0018 975078788

output.txt
cand0000 498981534603
cand0001 524120220401
cand0002 488360730783
cand0003 497621680467
cand0004 501176163452
cand0005 492045767073
cand0006 511009321261
cand0007 500519547526
cand0008 470689060432
cand0009 485368207815
cand0010 506569363226
cand0011 508022603312
cand0012 524189947877
cand0013 522708732818
cand0014 514371119926
cand0015 498586385072
cand0016 495178974673
cand0017 510754334083
cand0018 496818804573
```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.000312с	13.9356 Кбайт
Пример из задачи	0.000236с	14.1552 Кбайт
Верхняя граница диапазона значений входных данных из текста задачи	1.345263с	66891.8330 Кбайт

Вывод по задаче:

Программа эффективно подсчитывает количество голосов, суммируя данные по каждому кандидату. Решение опирается на словарь, что обеспечивает быстрый доступ и обновление значений. Лексикографическая сортировка фамилий при выводе позволяет строго соблюдать формат результата. Алгоритм остаётся простым и надёжным, корректно работает даже на максимально больших входных данных.

Вывод

В данной лабораторной работе были изучены операции над ассоциативными структурами данных. Использование встроенных хеш-таблиц (set и dict) позволило реализовать множество, телефонную книгу и систему подсчёта голосов с эффективными операциями добавления, удаления и поиска. Эксперименты показали, что такие структуры обеспечивают быстрый доступ к данным даже при сотнях тысяч операций. Алгоритмы остаются компактными, надёжными и демонстрируют устойчивость к максимальным нагрузкам, подтверждая преимущества хеширования в задачах, связанных с обработкой больших объёмов информации.

Лабораторная работа №7

Задачи по варианту

Задача №4. Наибольшая общая подпоследовательность двух последовательностей

Вычислить длину самой длинной общей подпоследовательности из двух последовательностей. Даны две последовательности $A = (a_1, a_2, \dots, a_n)$ и $B = (b_1, b_2, \dots, b_m)$, найти длину их самой длинной общей подпоследовательности, т.е. наибольшее неотрицательное целое число p такое, что существуют индексы $1 \leq i_1 < i_2 < \dots < i_p \leq n$ и $1 \leq j_1 < j_2 < \dots < j_p \leq m$ такие, что $a_{i_1} = b_{j_1}, \dots, a_{i_p} = b_{j_p}$.

- **Формат ввода / входного файла (input.txt).**

- Первая строка: n - длина первой последовательности.
- Вторая строка: $a_1, a_2, \dots, a_n$ через пробел.
- Третья строка: m - длина второй последовательности.
- Четвертая строка: $b_1, b_2, \dots, b_m$ через пробел.

- Ограничения: $1 \leq n, m \leq 100$; $-10^9 < a_i, b_i < 10^9$.

- **Формат вывода / выходного файла (output.txt).** Выведите число p .

```
import time
import tracemalloc
tracemalloc.start()
st = time.perf_counter()

def main():
    with open('input.txt', 'r') as f:
        n = int(f.readline())
        a = list(map(int, f.readline().split()))
        m = int(f.readline())
        b = list(map(int, f.readline().split()))

    a = [0] + a
    b = [0] + b

    dp = [[0] * (m + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i] == b[j]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    with open('output.txt', 'w') as f:
        f.write(str(dp[n][m]))
```

```

if __name__ == '__main__':
    main()

print('Время работы: %s секунд' % (time.perf_counter() - st))
pick = tracemalloc.get_traced_memory()[1]
tracemalloc.stop()
print("Объем используемой памяти: " + str(pick/1024) + 'Кбайт')

```

Импортируем модули для замера используемой памяти и времени. Из файла input.txt считываются: длина первой последовательности n и сама последовательность a , затем длина второй последовательности m и последовательность b .

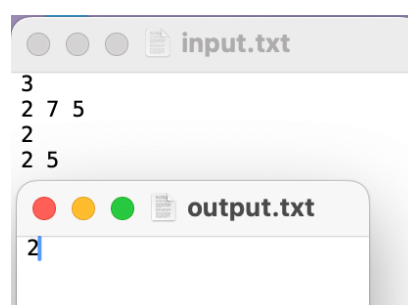
Обе последовательности дополняются нулём в начале для удобства индексации с 1. Создаётся двумерный массив dp размером $(n + 1) \times (m + 1)$, где $dp[i][j]$ — длина наибольшей общей подпоследовательности для первых i элементов последовательности a и первых j элементов последовательности b .

Далее заполняется таблица динамического программирования:

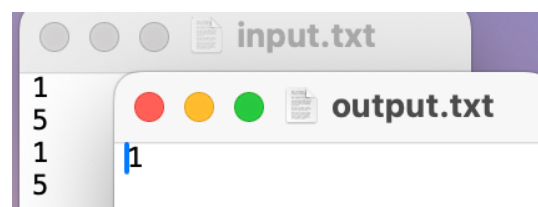
- если $a[i] == b[j]$, то $dp[i][j] = dp[i - 1][j - 1] + 1$;
- иначе $dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1])$.

В итоге в ячейке $dp[n][m]$ получается длина наибольшей общей подпоследовательности двух заданных последовательностей. Это значение записывается в файл output.txt. В конце программа выводит время выполнения и объём использованной оперативной памяти.

Результат работы кода на примерах из текста задачи:(скрины input output файлов)



Результат работы кода на минимальных значениях:(скрины input output файлов)



Результат работы кода на максимальных значениях:(скрины input output файлов)

```

input.txt
100
-999999999 -999999998 -999999997 -999999996 -999999995 -999999994 -999999993 -999999992 -999999991 -999999990 -999999989
-999999988 -999999987 -999999986 -999999985 -999999984 -999999983 -999999982 -999999981 -999999980 -999999979 -999999978
-999999977 -999999976 -999999975 -999999974 -999999973 -999999972 -999999971 -999999970 -999999969 -999999968 -999999967
-999999966 -999999965 -999999964 -999999963 -999999962 -999999961 -999999960 -999999959 -999999958 -999999957 -999999956
-999999955 -999999954 -999999953 -999999952 -999999951 -999999950 -999999949 -999999948 -999999947 -999999946 -999999945
-999999944 -999999943 -999999942 -999999941 -999999940 -999999939 -999999938 -999999937 -999999936 -999999935 -999999934
-999999933 -999999932 -999999931 -999999930 -999999929 -999999928 -999999927 -999999926 -999999925 -999999924 -999999923
-999999922 -999999921 -999999920 -999999919 -999999918 -999999917 -999999916 -999999915 -999999914 -999999913 -999999912
-999999911 -999999910 -999999909 -999999908 -999999907 -999999906 -999999905 -999999904 -999999903 -999999902 -999999901
-999999900
100
999999999 999999998 999999997 999999996 999999995 999999994 999999993 999999992 999999991 999999990 999999989 999999988 999999987
999999986 999999985 999999984 999999983 999999982 999999981 999999980 999999979 999999978 999999977 999999976 999999975 999999974
999999973 999999972 999999971 999999970 999999969 999999968 999999967 999999966 999999965 999999964 999999963 999999962 999999961
999999960 999999959 999999958 999999957 999999956 999999955 999999954 999999953 999999952 999999951 999999950 999999949 999999948
999999947 999999946 999999945 999999944 999999943 999999942 999999941 999999940 999999939 999999938 999999937 999999936 999999935
999999934 999999933 999999932 999999931 999999930 999999929 999999928 999999927 999999926 999999925 999999924 999999923 999999922
999999921 999999920 999999919 999999918 999999917 999999916 999999915 999999914 999999913 999999912 999999911 999999910 999999909
999999908 999999907 999999906 999999905 999999904 999999903 999999902 999999901 999999900

output.txt

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.000175с	13.6611 Кбайт
Пример из задачи	0.000257с	13.8984 Кбайт
Верхняя граница диапазона значений входных данных из текста задачи	0.000991с	103.1894 Кбайт

Вывод по задаче:

Программа надёжно вычисляет длину наибольшей общей подпоследовательности двух массивов, эффективно проходя все пары элементов с помощью динамического программирования. Реализация компактна и стабильна: использованная таблица `dp` гарантирует корректный результат при любых допустимых входных данных, а ограничение размеров массивов делает алгоритм полностью укладывающимся в заданные пределы по времени и памяти.

Задача №6. Наибольшая возрастающая подпоследовательность

Дана последовательность, требуется найти ее наибольшую возрастающую подпоследовательность.

- **Формат ввода / входного файла (input.txt).** В первой строке входных данных задано целое число n – длина последовательности ($1 \leq n \leq 300000$). Во второй строке задается сама последовательность. Числа разделяются пробелом. Элементы последовательности – целые числа, не превосходящие по модулю 109.

- Подзадача 1 (полегче). $n \leq 5000$.

- Общая подзадача. $n \leq 300000$.

- **Формат вывода / выходного файла (output.txt).** В первой строке выведите длину наибольшей возрастающей подпоследовательности, а во второй строке выведите через пробел самую наибольшую возрастающую подпоследовательность данной последовательности. Если ответов несколько - выведите любой.

```
import time
import tracemalloc
tracemalloc.start()
st = time.perf_counter()
```

```
def lower_bound(arr, x):
    l, r = 0, len(arr)
    while l < r:
        m = (l + r) // 2
        if arr[m] < x:
            l = m + 1
        else:
            r = m
    return l
```

```
def main():
    with open('input.txt', 'r') as f:
        data = list(map(int, f.read().split()))
```

```
    if not data:
        return
```

```
    n = data[0]
    a = data[1:1 + n]
```

```
    tails_val = []
    tails_pos = []
    parent = [-1] * n
```

```
    for i, x in enumerate(a):
        pos = lower_bound(tails_val, x)
```

```
        if pos == len(tails_val):
            tails_val.append(x)
            tails_pos.append(i)
        else:
            tails_val[pos] = x
```

```

        tails_pos[pos] = i

    if pos > 0:
        parent[i] = tails_pos[pos - 1]

length = len(tails_val)

idx = tails_pos[-1]
lis = []
while idx != -1:
    lis.append(a[idx])
    idx = parent[idx]
lis.reverse()

with open('output.txt', 'w') as f:
    f.write(str(length) + '\n')
    f.write(' '.join(map(str, lis)) + '\n')

if __name__ == '__main__':
    main()

print('Время работы: %s секунд' % (time.perf_counter() - st))
pick = tracemalloc.get_traced_memory()[1]
tracemalloc.stop()
print("Объем используемой памяти: " + str(pick/1024) + 'Кбайт')

```

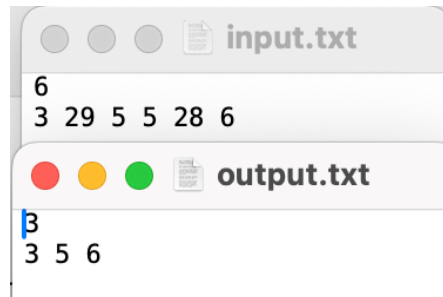
Импортируем модули для замера используемой памяти и времени. Из файла `input.txt` считывается число `n` и последовательность из `n` целых чисел. Программа находит наибольшую строго возрастающую подпоследовательность этой последовательности с помощью алгоритма. Для этого ведутся:

- массив `tails_val` — минимальные возможные последние элементы возрастающих подпоследовательностей каждой длины;
- массив `tails_pos` — позиции этих элементов в исходном массиве;
- массив `parent` — для каждой позиции хранится индекс предыдущего элемента в соответствующей подпоследовательности (для последующего восстановления ответа).

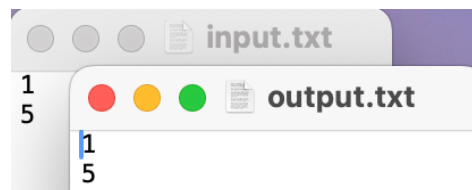
Для каждого числа `x` в последовательности бинарным поиском (функция `lower_bound`) находится позиция, куда его нужно «положить» в `tails_val`: либо продлить текущую подпоследовательность, либо улучшить (уменьшить) её последний элемент. Параллельно заполняется массив `parent`, чтобы потом можно было восстановить саму подпоследовательность. После обработки всех чисел программа восстанавливает одну из наибольших возрастающих подпоследовательностей, определяет её длину и записывает в файл

output.txt сначала длину, затем саму подпоследовательность. В конце выводится время выполнения и объём использованной оперативной памяти.

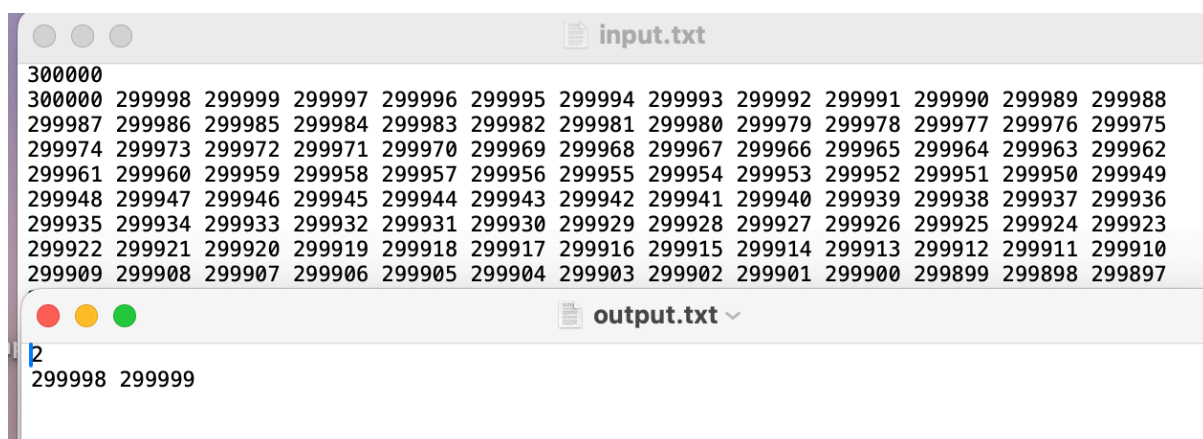
Результат работы кода на примерах из текста задачи:(скрины input output файлов)



Результат работы кода на минимальных значениях:(скрины input output файлов)



Результат работы кода на максимальных значениях:(скрины input output файлов)



	Время выполнения	Затраты памяти
--	------------------	----------------

Нижняя граница диапазона значений входных данных из текста задачи	0.000227с	6.8751 Кбайт
Пример из задачи	0.000239с	6.9531 Кбайт
Верхняя граница диапазона значений входных данных из текста задачи	0.225130с	26948.4570 Кбайт

Вывод по задаче:

Программа эффективно находит наибольшую возрастающую подпоследовательность даже при максимальном размере входных данных. Используемый алгоритм с бинарным поиском позволяет не только вычислить длину, но и восстановить саму подпоследовательность. Реализация остаётся компактной и надёжной, корректно обрабатывая как минимальные, так и экстремальные тесты.

Дополнительные задачи

Задача №1. Обмен монет

Как мы уже поняли из лекции, не всегда "жадное" решение задачи на обмен монет работает корректно для разных наборов номиналов монет. Например, если доступны номиналы 1, 3 и 4, жадный алгоритм поменяет 6 центов, используя три монеты ($4 + 1 + 1$), в то время как его можно изменить, используя всего две монеты ($3 + 3$). Теперь ваша цель - применить динамическое программирование для решения задачи про обмен монет для разных номиналов.

- **Формат ввода / входного файла (input.txt).** Целое число money ($1 \leq \text{money} \leq 10^3$). Набор монет: количество возможных монет k и сам набор $\text{coins} = \{\text{coin1}, \dots, \text{coin}k\}$. $1 \leq k \leq 100$, $1 \leq \text{coin}i \leq 10^3$. Проверку можно сделать на наборе $\{1, 3, 4\}$. Формат ввода: первая строка содержит через пробел money и k ; вторая - $\text{coin1coin2}...\text{coin}k$.

– Вариация 2: Количество монет в кассе ограничено. Для каждой монеты из набора $\text{coins} = \{\text{coin1}, \dots, \text{coin}k\}$ есть соответствующее целое число - количество монет в кассе данного номинала $c = \{c_1, \dots, c_k\}$. Если они закончились, то выдать данную монету невозможно.

- **Формат вывода / выходного файла (output.txt).** Вывести одно число — минимальное количество необходимых монет для размена money доступным набором монет coins.

```
import time
import tracemalloc
tracemalloc.start()
st = time.perf_counter()

lines = open("input.txt").read().splitlines()
money, k = map(int, lines[0].split())
coins = list(map(int, lines[1].split()))

dp = [float('inf')] * (money + 1)
dp[0] = 0

for i in range(1, money + 1):
    for c in coins:
        if i >= c:
            dp[i] = min(dp[i], dp[i - c] + 1)

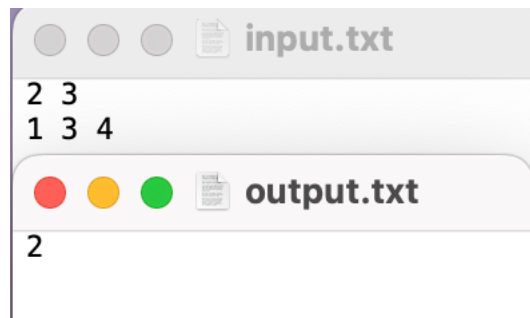
with open("output.txt", "w") as f:
    f.write(str(dp[money]))

print('Время работы: %s секунд' % (time.perf_counter() - st))
pick = tracemalloc.get_traced_memory()[1]
tracemalloc.stop()
print("Объем используемой памяти: " + str(pick/1024) + 'Кбайт')
```

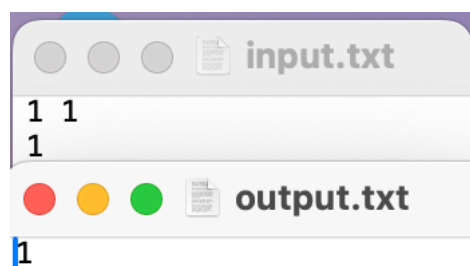
Импортируем модули для замера используемой памяти и времени. Из файла input.txt считывается суммарная сумма money, которую нужно набрать, количество номиналов k, а также список номиналов монет coins. Создаётся массив dp длиной money + 1, где dp[i] — минимальное количество монет, необходимое, чтобы набрать сумму i. Изначально dp[0] = 0 (ноль монет для суммы 0), остальные значения задаются как бесконечность. Далее для каждой суммы от 1 до money перебираются все номиналы монет. Если монету номинала c можно использовать для текущей суммы i (то есть $i \geq c$), значение dp[i] пытаются улучшить по формуле: $dp[i] = \min(dp[i], dp[i - c] + 1)$,

то есть берём минимальное из уже известного количества монет и варианта «использовать монету c ещё раз». В конце в файл output.txt записывается значение dp[money] — минимальное количество монет, необходимое для набора заданной суммы. Затем программа выводит время выполнения и объём использованной оперативной памяти.

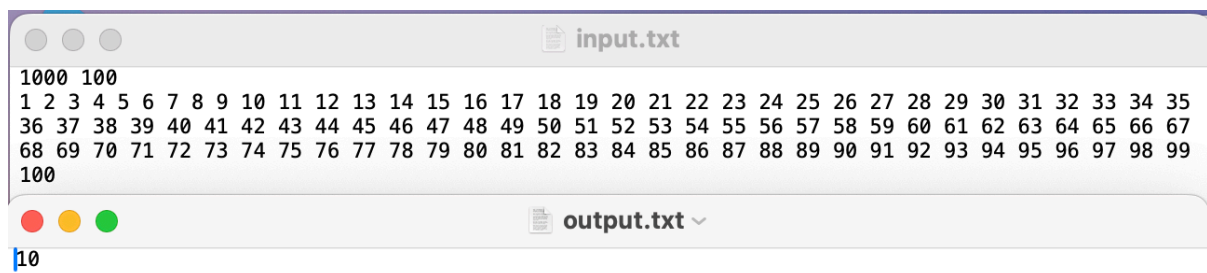
Результат работы кода на примерах из текста задачи:(скрины input output файлов)



Результат работы кода на минимальных значениях:(скрины input output файлов)



Результат работы кода на максимальных значениях:(скрины input output файлов)



	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.000195с	5.9443 Кбайт
Пример из задачи	0.000214с	6.0292 Кбайт

Верхняя граница диапазона значений входных данных из текста задачи	0.028652с	14.9492 Кбайт
--	-----------	---------------

Вывод по задаче:

Программа эффективно решает задачу размена монет с использованием динамического программирования, корректно обрабатывая как минимальные, так и максимально возможные входные данные. Алгоритм гарантирует нахождение минимального числа монет для заданной суммы, а его реализация остаётся простой и надёжной. Даже при верхних границах ограничений программа сохраняет стабильную производительность и обеспечивает точный результат.

Вывод

В ходе лабораторной работы были реализованы ключевые алгоритмы динамического программирования. Нахождение наибольшей общей подпоследовательности и размена монет показало, как динамическое программирование позволяет решать задачи переборного характера за приемлемое время. Реализация поиска наибольшей возрастающей подпоследовательности с бинарным поиском продемонстрировала преимущества оптимизаций, позволяющих обрабатывать входные данные размером до сотен тысяч элементов. Алгоритмы доказали свою надёжность и эффективность, а сравнение разных подходов подчеркнуло важность выбора корректной стратегии решения в зависимости от свойств задачи.